

Apunte de ROOT

Mauricio Santibañez

Diciembre 2024

Consideraciones Iniciales

Para este apunte se utilizará el editor de código **GEANY** como herramienta principal para el desarrollo y ejecución de los ejemplos. No se incluirán instrucciones sobre la instalación ni configuración inicial del entorno ROOT, centrándonos exclusivamente en su uso.

En esta primera versión del apunte, no se incluirán imágenes referenciales de los histogramas, gráficos o salidas de los programas. Se prioriza el desarrollo de los conceptos y la implementación de los códigos para facilitar su entendimiento y práctica.

1. ¿Qué es ROOT?

ROOT es una plataforma diseñada para el análisis de datos científicos, desarrollada por el **CERN**. Aunque su uso principal está orientado a la física de partículas, también es ampliamente utilizada en otras áreas que requieren el manejo y análisis eficiente de grandes volúmenes de datos.

En este apunte, nos centraremos en el uso de ROOT en sistemas basados en **Linux**.

Principales Características

- **Análisis de grandes cantidades de datos:** ROOT permite manejar de forma eficiente archivos de gran tamaño mediante su estructura jerárquica de datos, conocida como **TTree**.
- **Visualización:** Ofrece herramientas para crear histogramas, gráficos 2D y 3D con capacidades de visualización interactiva.
- **Compatibilidad con lenguajes:** Aunque está basado en C++, ROOT también es accesible desde Python mediante la biblioteca **pyROOT**.
- **Portabilidad:** Es compatible con los principales sistemas operativos, incluyendo Windows, macOS y Linux.

2. Exploración del entorno ROOT

Abrir ROOT

Para iniciar ROOT, abre una terminal en Linux y ejecuta el siguiente comando: **root**.

Esto abrirá la interfaz interactiva de ROOT, conocida como el *ROOT prompt*. Deberías ver un mensaje similar a este:

```
root [0]
```

Comandos básicos

A continuación, se presentan algunos comandos básicos que puedes ejecutar en el *ROOT prompt* para familiarizarte con su funcionamiento:

```
// Texto:
cout << "Hola, ROOT" << endl;

// Operaciones matematicas:
cout << 2 + 3 << endl;

// Histograma simple:
TCanvas *c1 = new TCanvas("c1", "Mi primer canvas", 800, 600);
TH1F *h1 = new TH1F("h1", "Histograma simple", 10, 0, 10);
h1->Fill(5);
h1->Draw();
```

Tarea práctica 1

Para consolidar lo aprendido, realiza las siguientes actividades:

1. Inicia ROOT en tu sistema y ejecuta los comandos mostrados anteriormente.
2. Crea un histograma con las siguientes características:
 - Llena el histograma con valores aleatorios entre 0 y 10.
 - Cambia el color de las barras del histograma.

Solución:

El siguiente código muestra cómo completar la tarea práctica:

```
void crearHistograma() {  
  
    // Crear un canvas para dibujar el histograma:  
    TCanvas *c1 = new TCanvas("c1", "Mi primer canvas", 800, 600);  
  
    // Crear histograma con 100 bins, valores entre 0 y 10:  
    TH1F *h1 = new TH1F("h1", "Histograma con valores aleatorios", 100, 0, 10);  
  
    // Llenar histograma con valores aleatorios entre 0 y 10:  
    for (int i = 0; i < 1000; i++) {  
        h1->Fill(gRandom->Uniform(0, 10));  
    }  
  
    // Cambiar color de las barras del histograma (color rojo):  
    h1->SetFillColor(kRed);  
  
    // Dibujar histograma:  
    h1->Draw();  
}
```

Para ejecutar el código desde Geany, sigue estos pasos:

1. Guarda el archivo con el nombre `crearHistograma.C`.
2. Abre una terminal en la carpeta donde guardaste el archivo.
3. Ejecuta el archivo en ROOT utilizando el siguiente comando: `root -l crearHistograma.C`.

Alternativamente, puedes ejecutar el archivo directamente dentro de ROOT escribiendo:

```
.x crearHistograma.C
```

3. Manejo de datos con ROOT

Estructuras de datos en ROOT: TFile, TTree y TBranch

ROOT está diseñado para manejar grandes volúmenes de datos mediante estructuras especializadas que optimizan el almacenamiento y el acceso eficiente. Las principales estructuras utilizadas son:

- **TFile:** Es el contenedor principal en ROOT. Permite almacenar y recuperar datos en formato binario dentro de un archivo con extensión `.root`. Este formato está optimizado para manejar grandes volúmenes de información.
- **TTree:** Es una estructura jerárquica diseñada para organizar datos de manera eficiente. Similar a una tabla en una base de datos, cada fila representa un evento o una entrada, mientras que cada columna corresponde a una variable o conjunto de datos.
- **TBranch:** Es una subdivisión de un **TTree**. Permite almacenar variables relacionadas dentro de un **TTree**, facilitando la lectura y escritura selectiva de datos. Esto mejora la eficiencia cuando solo se necesita acceder a partes específicas de un conjunto de datos.

TFile

El **TFile** es uno de los componentes fundamentales de ROOT, funcionando como un contenedor principal para almacenar y recuperar una amplia variedad de objetos utilizados en el análisis de datos, tales como histogramas, árboles (**TTree**), gráficos y otros elementos. Los archivos **TFile** tienen la extensión **.root**, un formato optimizado para manejar grandes volúmenes de datos de manera eficiente, tanto en términos de almacenamiento como de acceso.

Características principales

- **TFile** organiza los objetos ROOT en una estructura jerárquica, permitiendo la gestión de múltiples elementos dentro de un mismo archivo.
- Ofrece la capacidad de crear, leer, modificar y almacenar datos de manera persistente, facilitando la conservación y el acceso a los datos a lo largo del tiempo.
- Soporta diferentes modos de apertura, proporcionando flexibilidad para realizar diversas operaciones sobre los archivos, como lectura, escritura o modificación.

Ejemplo: Cómo abrir un archivo TFile

A continuación se muestra un ejemplo básico de cómo crear y abrir un archivo **TFile** en modo escritura. Este archivo puede luego contener objetos como histogramas o árboles.

```
TFile *file = TFile::Open("data.root", "RECREATE");
```

En este caso:

- **"data.root"** es el nombre del archivo que se creará o abrirá.
- **RECREATE** especifica que el archivo se creará desde cero, sobrescribiendo cualquier archivo existente con el mismo nombre.

Modos de apertura

Existen varios modos de apertura que permiten diferentes tipos de acceso al archivo **TFile**, según la operación que se desee realizar:

- **READ**: Abre el archivo en modo solo lectura, permitiendo el acceso a los datos sin modificarlos. Este modo es útil cuando solo se requiere analizar los datos.
- **UPDATE**: Abre el archivo en modo de lectura y escritura, permitiendo modificar los datos existentes. Es útil para actualizar archivos sin sobrescribirlos por completo.
- **RECREATE**: Crea un nuevo archivo y sobrescribe cualquier archivo existente con el mismo nombre, asegurando que los datos antiguos sean eliminados.

TTree

El **TTree** es uno de los objetos clave en ROOT, diseñado para almacenar grandes volúmenes de datos de manera eficiente. Se organiza jerárquicamente en **entradas** (también conocidas como eventos) y **ramas** (**branches**), donde cada rama puede contener una variable o conjunto de datos relacionados, y cada entrada representa un evento o una instancia de observación.

- El **TTree** es ideal para situaciones en las que es necesario almacenar grandes cantidades de datos estructurados, como en el análisis de datos experimentales en física de partículas, ya que permite manejar múltiples variables y eventos de manera eficiente.
- Los árboles (**TTree**) son más eficientes que los histogramas cuando se trata de almacenar y gestionar datos que consisten en varias variables por evento, ya que permiten un acceso más flexible y estructurado a la información.

Ejemplo de creación de un TTree

A continuación, se muestra un ejemplo de cómo crear un **TTree** en ROOT. En este ejemplo, el **TTree** se crea con un nombre y un título, los cuales permiten identificarlo dentro del archivo **TFile**.

```
TTree *tree = new TTree("eventTree", "Arbol de eventos");
```

En este caso:

- **eventTree** es el nombre del árbol, que debe ser único dentro del archivo **TFile**.
- **Arbol de eventos** es el título del árbol, que proporciona una breve descripción del contenido del árbol.

TBranch

El **TBranch** es una de las componentes fundamentales de un **TTree** en ROOT. Cada **TBranch** contiene los datos específicos correspondientes a una variable particular, y se organiza como una colección de valores que pertenecen a un evento. Mientras que el **TTree** organiza estos datos a nivel de eventos, las ramas (**TBranch**) representan las variables dentro de cada evento.

- Un **TBranch** se asocia a una variable específica, como el momento de una partícula (**px**) o la posición de un detector, y almacena el valor de esa variable para cada evento registrado en el **TTree**.
- Las ramas permiten almacenar múltiples variables de tipo diferente (enteros, flotantes, cadenas, etc.) dentro de un mismo **TTree**. Esta estructura jerárquica optimiza el acceso a grandes volúmenes de datos, lo que es esencial para análisis científicos de datos experimentales.
- En el contexto de la física de partículas, las ramas pueden almacenar datos provenientes de diversas mediciones de un experimento, como las energías, posiciones y momentos de partículas detectadas, permitiendo que cada evento se registre con un conjunto de características relevantes.
- Un **TBranch** facilita la lectura selectiva de los datos, lo que es útil cuando solo se necesita acceder a ciertas variables dentro de un evento, sin tener que cargar toda la estructura del **TTree**.

Ejemplo de cómo agregar una rama a un TTree

A continuación, se muestra un ejemplo de cómo agregar una rama a un **TTree** en ROOT. En este caso, se agrega una rama para almacenar el valor del momento en la dirección x de una partícula.

```
int px;  
tree->Branch("px", &px, "px/I");
```

En este ejemplo:

- **int px;** es la variable que almacena el valor del momento en la dirección x de la partícula. Esta variable tiene un tipo de dato entero (**int**).
- **"px"** es el nombre de la rama. Este nombre se utiliza para identificar la rama dentro del **TTree**.
- **&px** es el puntero a la variable que contiene los valores que serán almacenados en la rama. Este puntero permite que ROOT modifique directamente el valor de la variable durante el proceso de escritura de los datos.
- **"px/I"** especifica que la rama **px** almacenará valores de tipo entero (**int**, indicado por **I**).

Este proceso se repite para cada variable que se desee almacenar en el **TTree**, creando una estructura organizada de ramas, donde cada una representa una característica física o medición específica de cada evento.

Además de este ejemplo, se pueden agregar ramas para otros tipos de datos, como números de punto flotante (**D** para **Double_t**) o cadenas de caracteres (**C** para **Char_t**). Esto permite una gran flexibilidad al almacenar datos de experimentos complejos.

Operaciones básicas con TTree

Una de las principales ventajas de **TTree** es su capacidad para almacenar grandes volúmenes de datos de manera eficiente, permitiendo tanto la escritura como la lectura rápida. Los árboles en **ROOT** son especialmente útiles en situaciones donde se desea almacenar múltiples eventos, cada uno con varias variables. A continuación, se detallan los pasos básicos para crear, llenar y leer un **TTree**.

Crear, llenar y leer datos en árboles

El proceso de llenar un **TTree** con datos implica definir primero las ramas, luego asignar valores a estas ramas dentro de un bucle, y finalmente llenar el árbol con los datos correspondientes. De manera similar, la lectura de datos de un **TTree** se realiza accediendo a las ramas mediante punteros a variables y utilizando funciones específicas para iterar a través de las entradas del árbol.

Ejemplo para llenar un TTree

El siguiente código muestra cómo crear un **TTree**, definir ramas, y llenar el árbol con datos aleatorios.

```
int px, py;
TTree *tree = new TTree("eventTree", "Arbol de eventos");

// Definir ramas para el momento en x y y
tree->Branch("px", &px, "px/I"); // Define rama para px
tree->Branch("py", &py, "py/I"); // Define rama para py

// Llenar el TTree con datos aleatorios
for (int i = 0; i < 100; i++) {
    px = rand() % 100; // Genera valor aleatorio para px
    py = rand() % 100; // Genera valor aleatorio para py
    tree->Fill(); // Llenar TTree con datos generados
}
```

En este ejemplo:

- Se definen dos ramas: **px** y **py**, que almacenan valores enteros. El tipo de cada rama se especifica con la cadena **"px/I"** (entero para **px**) y **"py/I"** (entero para **py**).
- Dentro del bucle **for**, se asignan valores aleatorios a **px** y **py** utilizando la función **rand()**.
- La función **tree->Fill()** agrega un nuevo evento (con los valores de **px** y **py**) al **TTree** en cada iteración.

Este proceso puede ser fácilmente adaptado a datos experimentales o simulados, donde las ramas contienen información sobre distintas variables físicas, como momento, energía, o posición.

Ejemplo de cómo leer un TTree

Para leer datos de un **TTree** previamente almacenado, se abre el archivo que contiene el **TTree**, se obtiene el **TTree** dentro del archivo y luego se asocian las ramas con las variables en el código. A continuación, se presenta un ejemplo para leer un **TTree** y mostrar los valores almacenados en las ramas.

```
TFile *file = TFile::Open("data.root", "READ");
TTree *tree = (TTree*)file->Get("eventTree");

// Asociar las ramas con las variables en el código
tree->SetBranchAddress("px", &px);
tree->SetBranchAddress("py", &py);

// Leer y mostrar los datos del TTree
for (int i = 0; i < tree->GetEntries(); i++) {
    tree->GetEntry(i);
    std::cout << "px: " << px << ", py: " << py << std::endl;
}
```

En este ejemplo:

- `TFile::Open("data.root", READ)` abre el archivo `data.root` en modo solo lectura. Esto permite acceder a los datos almacenados sin modificar el archivo.
- `tree->Get(eventTree)` obtiene el `TTree` con el nombre `eventTree` del archivo.
- `tree->SetBranchAddress("px", &px)` establece la asociación entre la rama `px` y la variable `px` en el código, lo que permite que los valores de la rama se almacenen en la variable `px` al leer los datos.
- `tree->GetEntry(i)` obtiene la entrada `i` del `TTree`, llenando las variables asociadas con los valores correspondientes de las ramas del evento.
- Finalmente, los valores de `px` y `py` se muestran en la consola utilizando `std::cout`.

Este proceso es muy útil cuando se desea realizar análisis sobre grandes cantidades de eventos, ya que ROOT optimiza el acceso a los datos a través de su sistema de ramas y permite trabajar eficientemente con datos experimentales o simulados.

TLeaf

En ROOT, un `TLeaf` representa una *hoja* dentro de una *rama* de un `TTree`. Mientras que las ramas (`TBranch`) organizan y almacenan los datos de un conjunto de variables, cada hoja de una rama corresponde a una variable individual de tipo simple (por ejemplo, `int`, `float`, `double`, etc.).

- Un `TLeaf` describe cómo se organizan los datos de una rama específica. Cada variable dentro de una rama, como `px` o `py`, está asociada con un `TLeaf`.
- Los `TLeaf` permiten acceder de manera eficiente a los valores de las variables almacenadas en un `TTree`. Cada hoja contiene un valor único asociado a un evento específico en la rama de datos.
- Para obtener un `TLeaf` de una rama, se utiliza el método `GetLeaf()`. Este método facilita el acceso directo a los valores individuales de las variables almacenadas en la rama.

Ejemplo para acceder a un TLeaf:

En el siguiente ejemplo, se muestra cómo acceder a un `TLeaf` específico dentro de una rama y obtener su valor:

```
tree->GetEntry(0);  
TLeaf *leaf = tree->GetLeaf("px");  
std::cout << "Valor de px: " << leaf->GetValue() << std::endl;
```

En este ejemplo:

- `tree->GetEntry(0)` carga la entrada correspondiente al primer evento del `TTree`. Este paso es opcional si el evento ya está cargado en memoria.
- `TLeaf *leaf = tree->GetLeaf("px");` obtiene el `TLeaf` asociado a la variable `px` dentro de la rama correspondiente.
- `leaf->GetValue()` devuelve el valor de la variable almacenada en la hoja. Este valor corresponde al valor de `px` en el evento cargado en memoria.

El uso de `TLeaf` facilita el acceso directo a las variables de un `TTree`, mejorando la eficiencia al trabajar con grandes volúmenes de datos, ya que permite acceder y manipular las variables de forma rápida y precisa sin tener que recorrer todo el `TTree`.

Exploración de datos: Comandos Show, Print y Scan

Una vez que los datos han sido cargados en un `TTree`, ROOT proporciona varios comandos interactivos para explorar y visualizar los datos de manera eficiente. Estos comandos permiten obtener información detallada sobre los eventos almacenados y la estructura del árbol de datos.

Comando Show

El comando **Show** permite visualizar la información de una entrada específica de un **TTree**. Es útil cuando se desea inspeccionar los valores de todas las ramas para un evento particular.

Sintaxis:

```
tree->Show(entryNumber);
```

Ejemplo:

```
tree->Show(0);
```

Este comando imprime todos los valores de las ramas del **TTree** para el evento especificado por **entryNumber**. En este caso, el número 0 hace referencia al primer evento almacenado en el **TTree**.

Comando Print

El comando **Print** ofrece una visión general de la estructura completa del **TTree**. Muestra todas las ramas y sus características, proporcionando una descripción detallada de las variables y cómo están organizadas dentro del árbol.

Sintaxis:

```
tree->Print();
```

Este comando es útil para obtener una visión global del **TTree**, permitiendo verificar rápidamente qué ramas contiene, sus tipos de datos y otras propiedades asociadas a la estructura del árbol.

Comando Scan

El comando **Scan** es una herramienta poderosa para explorar varias columnas de datos de diferentes ramas de un **TTree** al mismo tiempo. Es similar a realizar un "escaneo" o "listado" de los valores almacenados en las ramas seleccionadas.

Sintaxis:

```
tree->Scan("branch1:branch2:branch3");
```

Ejemplo:

```
tree->Scan("px:py");
```

En este ejemplo, el comando **Scan** muestra los valores de las ramas **px** y **py** para todos los eventos almacenados en el **TTree**. Es una forma rápida y eficiente de inspeccionar múltiples columnas de datos de una vez.

Estos comandos facilitan la exploración y el análisis de los datos, permitiendo una interacción rápida y eficaz con grandes volúmenes de información almacenados en los **TTree**.

Visualización con TTree::Draw

Una de las características más poderosas de ROOT es la capacidad de visualizar datos de un **TTree** de manera fácil y rápida. El comando **TTree::Draw** permite crear gráficos directamente a partir de los datos almacenados, lo que facilita el análisis visual de los mismos.

El tipo de gráfico generado depende de cómo se estructuran los datos y de las opciones especificadas en el comando.

Sintaxis básica de TTree::Draw

```
tree->Draw("expresion", "condicion", "opciones");
```

- **expresión**: Especifica qué datos graficar. Puede ser una rama del **TTree** o una operación matemática entre ramas, por ejemplo: "px", "px+py", **energia**.

- **condición (opcional):** Permite aplicar un filtro para seleccionar solo los eventos que cumplan una condición lógica, por ejemplo: `"px > 0"`.
- **opciones (opcional):** Define cómo se visualizan los datos (el estilo del gráfico). Se puede elegir entre varias opciones predefinidas.

Tipos de gráficos que se pueden generar con `TTree::Draw`

El comando `TTree::Draw` puede generar diferentes tipos de gráficos en función de cómo se especifiquen las expresiones y las opciones. A continuación, se describen los gráficos más comunes:

1. **Gráfico de dispersión (Scatter plot):** Cuando seleccionas dos variables en la expresión, ROOT genera un gráfico de dispersión.

Ejemplo:

```
tree->Draw("px:py");
```

Esto crea un gráfico de dispersión donde `px` se coloca en el eje X y `py` en el eje Y.

2. **Histograma:** Para generar un histograma, se utiliza el operador `>>` para especificar el nombre del histograma, así como el número de bins y el rango del eje X.

Ejemplo:

```
tree->Draw("px >> hist(100, 0, 10)");
```

Esto genera un histograma de la rama `px`, con 100 bins y valores entre 0 y 10. Aquí, `hist` es el nombre del histograma.

3. **Histograma de dos dimensiones:** Si seleccionas dos variables en la expresión y especificas un rango, ROOT genera un histograma bidimensional.

Ejemplo:

```
tree->Draw("px:py >> hist2D(100, 0, 10, 100, 0, 10)");
```

Esto crea un histograma 2D donde `px` está en el eje X y `py` en el eje Y, con 100 bins en cada eje y un rango de 0 a 10.

4. **Gráfico de barras:** Aunque los histogramas generados por defecto ya tienen forma de barras, se puede forzar este estilo con la opción `"HIST"`.

Ejemplo:

```
tree->Draw("px >> hist(50, 0, 100)", "", "HIST");
```

El uso de la opción `"HIST"` asegura que el gráfico sea un histograma de barras.

5. **Extras:** El parámetro `opciones` permite personalizar aún más los gráficos. Algunas de las opciones más comunes incluyen:

- **HIST:** Crea un histograma de barras.
- **COLZ:** Genera un gráfico 2D con escala de colores.
- **P:** Dibuja un gráfico de puntos.
- **A:** Añade datos a un gráfico existente.
- **E:** Dibuja el histograma con barras de error.

Ejemplo:

```
tree->Draw("px:py >> hist2D(100, 0, 10, 100, 0, 10)", "", "COLZ");
```

Este comando genera un histograma 2D con colores para representar la densidad de los datos.

Con estas capacidades, `TTree::Draw` se convierte en una herramienta esencial para explorar y analizar datos en ROOT.

A considerar:

Cuando no necesitas aplicar ninguna condición, simplemente dejas el segundo argumento vacío (). Esto le indica a ROOT que no filtre eventos y utilice todos los datos disponibles. Ejemplo:

```
tree->Draw("px", "", "HIST");
```

Tarea práctica 2

1. Crear un `TTree` con ramas `px`, `py` y `energy`.
2. Llenar el `TTree` con 100 eventos, generando valores aleatorios.
3. Acceder a los valores de las ramas utilizando `TLeaf` para imprimir el valor de `px` y `py` del primer evento.
4. Usando `TTree::Draw`, generar:
 - Un histograma de `px`.
 - Un gráfico de dispersión de `px` vs `py`.
 - Un histograma 2D de `px` y `py`.

Solución:

A continuación, se muestra el código completo para la tarea práctica. Este script en C++ crea un archivo ROOT que almacena un `TTree` con los valores aleatorios de las variables `px`, `py` y `energy`, y luego genera varios gráficos a partir de esos datos:

```
void tarea2() {  
  
    // Crear TFile para almacenar TTree  
    TFile *file = new TFile("datos.root", "RECREATE");  
  
    // Crear un TTree  
    TTree *tree = new TTree("eventTree", "Arbol de eventos");  
  
    // Variables para las ramas  
    int px, py;  
    float energy;  
  
    // Crear ramas  
    tree->Branch("px", &px, "px/I");  
    tree->Branch("py", &py, "py/I");  
    tree->Branch("energy", &energy, "energy/F");  
  
    // Llenar el TTree con datos aleatorios  
    for (int i = 0; i < 100; i++) {  
        px = rand() % 100; // Valores aleatorios px  
        py = rand() % 100; // Valores aleatorios py  
        energy = rand() % 100; // Valores aleatorios energy  
        tree->Fill(); // Llenar el TTree con datos generados  
    }  
}
```

```

}

// Guardar el archivo ROOT
file->Write();
file->Close();

// A partir de aqui, se puede ejecutar el script directamente en ROOT con el
// comando:
// root -l tarea2.C

// Ahora abrimos el archivo y accedemos al TTree
file = TFile::Open("datos.root", "READ");
tree = (TTree*)file->Get("eventTree");

// Acceder a las ramas usando TLeaf
TLeaf *leaf_px = tree->GetLeaf("px");
TLeaf *leaf_py = tree->GetLeaf("py");

// Cargar el primer evento del TTree
tree->GetEntry(0);

// Imprimir los valores de px y py del primer evento
std::cout << "Valor de px en el primer evento: " << leaf_px->GetValue() <<
std::endl;
std::cout << "Valor de py en el primer evento: " << leaf_py->GetValue() <<
std::endl;

// 1. Histograma de px
TCanvas *c1 = new TCanvas("c1", "Graficos de ROOT", 800, 600);
tree->Draw("px >> hist(50, 0, 100)", "", "HIST");

// 2. Grafico de dispersion px vs py
TCanvas *c2 = new TCanvas("c2", "Grafico de dispersion", 800, 600);
tree->Draw("px:py", "", "COLZ");

// 3. Histograma 2D de px vs py
TCanvas *c3 = new TCanvas("c3", "Histograma 2D", 800, 600);
tree->Draw("px:py >> hist2D(100, 0, 100, 100, 0, 100)", "", "COLZ");
}

```

Este código realiza los siguientes pasos:

- Primero, crea un archivo `TFile` llamado `datos.root` y dentro de él un `TTree` denominado `eventTree`.
- Luego, define las ramas `px`, `py` y `energy`, y las llena con valores aleatorios en un bucle de 100 eventos.
- Tras completar la carga de los datos, guarda el archivo ROOT y lo cierra.
- Posteriormente, se vuelve a abrir el archivo para acceder al `TTree`, y se utiliza `TLeaf` para acceder a los valores de las ramas `px` y `py`.
- Finalmente, se generan tres gráficos con el comando `TTree::Draw`: un histograma de `px`, un gráfico de dispersión entre `px` y `py`, y un histograma 2D de `px` y `py`.

4. Gráficos y análisis básico

Creación y personalización de histogramas

En ROOT, un histograma es un objeto que organiza los datos en intervalos (denominados **bins**) y cuenta cuántos valores caen dentro de cada intervalo. Los histogramas son herramientas fundamentales en el análisis de datos experimentales, ya que permiten visualizar la distribución de los datos, su frecuencia y realizar un análisis estadístico adecuado. A través de estos gráficos, es posible identificar patrones, anomalías y realizar ajustes de funciones que describan los datos.

Tipos de histogramas en ROOT

ROOT ofrece una variedad de clases de histogramas, que permiten trabajar con diferentes tipos de datos. Los histogramas se pueden clasificar en función de su dimensionalidad y el tipo de datos que almacenan. A continuación, se describen los tipos principales:

1. TH1: Histogramas unidimensionales.

- TH1F: Histograma de tipo flotante (**float**), adecuado para almacenar datos con precisión de punto flotante de 32 bits.
- TH1D: Histograma de doble precisión (**double**), utilizado para datos con mayor precisión numérica (64 bits).
- TH1I: Histograma de tipo entero (**int**), usado para almacenar valores enteros.

2. TH2: Histogramas bidimensionales.

- TH2F: Histograma 2D con datos flotantes (**float**).
- TH2D: Histograma 2D con datos de doble precisión (**double**).
- TH2I: Histograma 2D con datos enteros (**int**).

Creación de histogramas

Crear un histograma 1D (TH1F)

Para crear un histograma unidimensional en ROOT, se utiliza la clase **TH1F**, que permite almacenar datos de tipo flotante. A continuación se muestra un ejemplo de cómo se crea un histograma 1D:

```
TH1F *h1 = new TH1F("h1", "Histograma 1D", 50, 0, 10);
```

En este ejemplo, se especifican los siguientes parámetros:

- **h1**: Nombre del histograma. Este identificador es utilizado para referirse al histograma en el código.
- **"Histograma 1D"**: Título del histograma, el cual se muestra en la leyenda del gráfico generado.
- **50**: Número de **bins** o intervalos que dividirán el rango de valores en el eje X. Este parámetro determina la resolución del histograma.
- **0, 10**: Rango de los valores en el eje X. En este caso, los valores del histograma estarán distribuidos en el intervalo $[0, 10]$.

Crear un histograma 2D (TH2F)

Para crear un histograma bidimensional en ROOT, se utiliza la clase **TH2F**, que permite almacenar datos de tipo flotante en dos dimensiones. A continuación se muestra un ejemplo de cómo se crea un histograma 2D:

```
TH2F *h2 = new TH2F("h2", "Histograma 2D", 50, 0, 10, 50, 0, 10);
```

En este ejemplo, se especifican los siguientes parámetros:

- **h2**: Nombre del histograma. Este identificador es utilizado para referirse al histograma en el código.

- "Histograma 2D": Título del histograma, que será mostrado en la leyenda del gráfico generado.
- 50, 0, 10: Configuración del eje X. El primer número (50) indica el número de `bins`, y el segundo (0) y tercero (10) definen el rango de valores en el eje X, es decir, el histograma abarcará valores entre 0 y 10 en este eje.
- 50, 0, 10: Configuración del eje Y. Al igual que en el eje X, el primer número (50) representa el número de `bins` para el eje Y, y el segundo (0) y tercero (10) definen el rango de valores en el eje Y, con un intervalo de valores entre 0 y 10.

Llenado de histogramas

El llenado de histogramas en ROOT se realiza mediante el método `Fill()`, el cual agrega valores o puntos a los bins del histograma según los datos proporcionados.

Llenado manual

Para llenar un histograma de manera manual, se puede utilizar el método `Fill()` especificando los valores correspondientes. A continuación, se muestran algunos ejemplos:

```
h1->Fill(3.5);
h2->Fill(2.1, 4.3);
```

En el primer ejemplo, el valor 3.5 es agregado al histograma unidimensional `h1`. En el segundo ejemplo, el punto bidimensional (2.1, 4.3) es agregado al histograma `h2` de dos dimensiones.

Llenado automático

Para llenar un histograma de forma automática, se puede utilizar un bucle junto con una función generadora de números aleatorios. A continuación, se muestra un ejemplo de cómo llenar un histograma con valores aleatorios generados dentro de un rango:

```
for (int i = 0; i < 1000; i++) {
    h1->Fill(gRandom->Uniform(0, 19));
}
```

En este ejemplo, el bucle `for` itera 1000 veces, y en cada iteración se genera un valor aleatorio con `gRandom->Uniform(0, 19)` dentro del rango [0, 19], que luego es agregado al histograma `h1`.

Personalización de histogramas

ROOT permite personalizar diversos aspectos visuales de los histogramas, como los colores, el estilo de las líneas, los títulos de los ejes, y más. Estas modificaciones pueden mejorar la presentación de los datos y facilitar la interpretación de los resultados.

Ejemplo de personalización de un histograma 1D (TH1F)

Para modificar los aspectos visuales de un histograma 1D, se utilizan métodos específicos que permiten ajustar el color de relleno, el color de las líneas, los títulos de los ejes y finalmente dibujar el histograma. A continuación, se muestra un ejemplo de cómo personalizar un histograma 1D:

```
h1->SetFillColor(kBlue); // Cambiar color relleno azul
h1->SetLineColor(kRed); // Cambiar color de linea a rojo
h1->GetXaxis()->SetTitle("X"); // titulo eje X
h1->GetYaxis()->SetTitle("Frecuencia"); // titulo eje Y
h1->Draw(); // Dibujar histograma
```

En este ejemplo, se cambia el color de relleno del histograma `h1` a azul utilizando `SetFillColor(kBlue)` y el color de la línea a rojo con `SetLineColor(kRed)`. Además, se asignan títulos a los ejes X e Y, y finalmente se dibuja el histograma con `Draw()`.

Ejemplo de personalización de un histograma 2D (TH2F)

La personalización de histogramas bidimensionales (TH2F) también es posible. En este caso, además de modificar los colores y títulos, se puede establecer un modo de visualización en colores. A continuación, se muestra un ejemplo de cómo personalizar un histograma 2D:

```
h2->SetFillColor(kGreen); // Cambiar color relleno verde
h2->GetXaxis()->SetTitle("Eje X"); // titulo eje X
h2->GetYaxis()->SetTitle("Eje Y"); // titulo eje Y
h2->Draw("COLZ"); // Dibujar histograma 2D en modo color
```

En este caso, el color de relleno del histograma `h2` se establece en verde con `SetFillColor(kGreen)`. Los títulos de los ejes X e Y también son modificados. Finalmente, se utiliza `Draw("COLZ")` para mostrar el histograma en una representación de colores (modo `COLZ`), donde el color de cada celda indica la frecuencia de los valores en esa región.

Ajuste de datos con funciones TF1

Una `TF1` es un objeto en ROOT que representa una función matemática unidimensional. Este tipo de función puede ser utilizada tanto para graficar funciones matemáticas como para ajustar datos experimentales. Las funciones `TF1` pueden ser predefinidas, como la función ‘gauss’ para una distribución gaussiana, o bien definidas por el usuario mediante expresiones matemáticas o funciones personalizadas en C++.

Las `TF1` son herramientas fundamentales para:

- **Graficar** funciones matemáticas, permitiendo representar visualmente expresiones algebraicas o analíticas.
- **Ajustar** datos experimentales, como los obtenidos de histogramas, mediante técnicas de ajuste no lineales o lineales.
- **Cálculos** basados en parámetros de ajustes, permitiendo extraer información cuantitativa de los datos experimentales, como valores óptimos para parámetros de ajuste y sus incertidumbres.

Creación de una TF1

Funciones predefinidas: ROOT ofrece varias funciones predefinidas que permiten un ajuste rápido de los datos, tales como `gaus` (gaussiana), `expo` (exponencial), y `poln` (polinomios), entre otras.

Ejemplo: Crear una función gaussiana

```
TF1 *func = new TF1("func", "gaus", 0, 10);
```

- `func`: Nombre del objeto de la función.
- `gaus`: Tipo de función predefinida (distribución gaussiana).
- `0, 10`: Rango en el eje X donde se define la función, en este caso de 0 a 10.

Funciones definidas por el usuario: Además de las funciones predefinidas, ROOT permite definir funciones personalizadas utilizando expresiones matemáticas o implementando código C++ para una mayor flexibilidad.

Ejemplo: Definir una función matemática personalizada

```
TF1 *func = new TF1("customFunc", "[0]*sin([1]*x)", 0, 10);
customFunc->SetParameters(1.0, 2.0); // [0] = 1.0, [1] = 2.0
```

- `[0]` y `[1]`: Representan los parámetros de la función. En este caso, `[0]` es un multiplicador y `[1]` controla la frecuencia de la función seno.
- `SetParameters`: Este método establece los valores iniciales para los parámetros de la función. En este ejemplo, `[0] = 1.0` y `[1] = 2.0`.

Graficar una TF1

Una vez definida una función, se puede graficar directamente en un `TCanvas` utilizando el método `Draw()`. Esto permite visualizar la función matemática en el rango deseado.

Ejemplo: Graficar una gaussiana

```
TF1 *func = new TF1("func", "gaus", 0, 10);  
func->Draw();
```

Ajuste de un histograma con una TF1

ROOT ofrece una forma sencilla de ajustar funciones a datos experimentales, minimizando la diferencia entre los valores del histograma y la función seleccionada. Este ajuste se realiza utilizando el método `Fit()`.

Ejemplo: Ajustar una función gaussiana a un histograma

```
// Crear un histograma y llenarlo de datos  
TH1F *h1 = new TH1F("h1", "Histograma", 50, 0, 10);  
for (int i = 0; i < 1000; i++) {  
    h1->Fill(gRandom->Gaus(5, 1));  
}  
  
// Crear una funcion gaussiana  
TF1 *func = new TF1("func", "gaus", 0, 10);  
  
// Ajustar la funcion al histograma  
h1->Fit("func");  
h1->Draw();
```

- `gRandom->Gaus(5, 1)`: Este método genera valores aleatorios distribuidos de acuerdo con una distribución gaussiana, con una media de 5 y una desviación estándar de 1.
- `h1->Fit("func")`: Ajusta la función `func` al histograma `h1` utilizando el método de mínimos cuadrados.

El ajuste de la función permite obtener parámetros clave, que son fundamentales para describir el comportamiento de los datos. Aunque estos parámetros son válidos para cualquier función, su significado variará dependiendo de la función utilizada y de los datos en cuestión.

Obtener parámetros de ajuste:

Una vez realizada la operación de ajuste, es posible extraer los parámetros de la función ajustada.

```
double mean = func->GetParameter(1); // media  
double sigma = func->GetParameter(2); // desviacion estandar
```

Calcular el valor de la función en un punto específico:

Para obtener el valor de la función ajustada en un punto específico, se utiliza el método `Eval()`.

```
double y = func->Eval(5.0); // Evala la funcion en x = 5.0
```

Integrar la función en un rango:

Es posible calcular la integral de la función en un intervalo dado mediante el método `Integral()`.

```
double integral = func->Integral(0, 10);
```

Tarea práctica 3

1. Ajusta un histograma con una función gaussiana (gaus):
 - Llena un histograma con datos aleatorios de distribución gaussiana.
 - Ajusta una función TF1 y muestra los parámetros ajustados (media y desviación estándar).
2. Crea y ajusta una función seno personalizada:
 - Llena un histograma con datos que sigan un patrón sinusoidal con ruido.
 - Ajusta la función seno personalizada y observa los resultados.

Solución:

Parte 1: Ajustar histograma con función gaussiana

```
void ajustarGaussiana() {
    TH1F *h1 = new TH1F("h1", "Ajuste Gaussiano", 50, 0, 10);

    // Llenar histograma con datos aleatorios de distribucion gaussiana
    for (int i = 0; i < 1000; i++) {
        h1->Fill(gRandom->Gaus(5,1));
    }

    // Crear una funcion gaussiana y ajustar al histograma
    TF1 *gausFunc = new TF1("gausFunc", "gaus", 0, 10);
    h1->Fit("gausFunc");

    // Obtener parametros ajustados
    double mean = gausFunc->GetParameter(1); // Media
    double sigma = gausFunc->GetParameter(2); // Desviacion estandar

    std::cout << "Media ajustada: " << mean << std::endl;
    std::cout << "Desviacion estandar: " << sigma << std::endl;

    // Dibujar el histograma ajustado
    h1->Draw();
}
```

Parte 2: Crear y ajustar una función seno personalizada

```
double miFuncion(double *x, double *parms) {
    double xx = x[0];
    double amplitud = parms[0]; // Amplitud
    double frequency = parms[1]; // Frecuencia
    return amplitud * sin(frequency * xx);
}

void ajustarFuncionSeno() {
    TH1F *h1 = new TH1F("h1", "Ajuste funcion seno", 50, 0, 10);

    // Llenar histograma con datos sinusoidales con ruido
    for (int i = 0; i < 1000; i++) {
        double x = gRandom->Uniform(0, 10);
        double ruido = gRandom->Gaus(0, 0.2); // ruido gaussiano
        double y = sin(2 * x) + ruido; // Seno con ruido
        h1->Fill(x, y);
    }

    // Crear y ajustar la funcion seno personalizada
    TF1 *senoFunc = new TF1("senoFunc", miFuncion, 0, 10, 2);
    senoFunc->SetParameters(1.0, 2.0); // Establecer parametros iniciales

    h1->Fit("senoFunc");

    // Obtener parametros ajustados
    double amplitud = senoFunc->GetParameter(0); // Amplitud
    double frequency = senoFunc->GetParameter(1); // Frecuencia

    std::cout << "Amplitud ajustada: " << amplitud << std::endl;
    std::cout << "Frecuencia ajustada: " << frequency << std::endl;

    // Dibujar el histograma ajustado
    h1->Draw();
}
```

Cómo ejecutar estas funciones:

1. Escribe ambas funciones en un archivo llamado ajustes.C.
2. Ejecuta cada una por separado en ROOT:

- Para el ajuste gaussiano:

```
root -l ajustes.C
root [0] ajustarGaussiana()
```

- Para el ajuste con la función seno personalizada:

```
root -l ajustes.C
root [0] ajustarFuncionSeno()
```

5. Análisis avanzado de datos

Filtros y cortes: Uso de TCut y expresiones lógicas

Cuando se trabaja con grandes volúmenes de datos en ROOT, a menudo se necesita seleccionar eventos específicos basándose en condiciones lógicas o criterios de selección (cortes). ROOT ofrece dos herramientas principales para realizar estas selecciones:

- **TCut**: Representa un corte lógico como una cadena de texto que puede ser utilizada para filtrar datos de forma eficiente.
- **Expresiones lógicas directas**: Se utilizan cuando se trabaja con comandos como `TTree::Draw` o al analizar los datos, permitiendo la creación de filtros más complejos.

TCut

Un **TCut** es un objeto de ROOT que encapsula una condición lógica. Este objeto puede reutilizarse en múltiples análisis, lo que simplifica y optimiza los procesos, ya que permite aplicar las mismas condiciones de corte en diferentes operaciones o análisis de datos.

Sintaxis básica:

```
TCut cut = "energy > 10 && px < 50";
```

`energy >10 && px <50` representa una condición lógica donde la energía debe ser mayor que 10 y el valor de `px` debe ser menor que 50. Este corte se puede aplicar a los datos de un **TTree** o cualquier otro conjunto de datos en ROOT, asegurando que solo los eventos que cumplan con ambas condiciones sean seleccionados para análisis posteriores.

Aplicar cortes en ROOT

Los cortes son fundamentales al trabajar con grandes conjuntos de datos, ya que permiten seleccionar únicamente los eventos que cumplen con ciertas condiciones. En ROOT, los cortes pueden aplicarse de manera eficiente mediante el uso de expresiones lógicas o el objeto **TCut**, que encapsula dichas condiciones.

Corte simple con TTree::Draw

El método **TTree::Draw** permite graficar ramas de un **TTree** aplicando cortes directamente en la expresión de dibujo. Esta es una forma sencilla y rápida de aplicar un filtro sobre los datos.

```
tree->Draw("px", "energy > 10");
```

- `px`: Rama que se graficará en el eje X.
- `energy >10`: Condición que deben cumplir los eventos seleccionados. Solo se graficarán los eventos con `energy` mayor a 10.

Este tipo de corte es útil para análisis rápidos, pero cuando los cortes se vuelven más complejos o se repiten a lo largo de varios análisis, es recomendable utilizar **TCut**.

Usar TCut para simplificar cortes

El uso de **TCut** permite crear condiciones de corte reutilizables y combinarlas de manera sencilla. Esto facilita la gestión de condiciones complejas y la reutilización en diferentes análisis o gráficos.

```
TCut cut1 = "energy > 10";  
TCut cut2 = "px < 50";  
TCut combinedCut = cut1 && cut2;  
  
tree->Draw("px:py", combinedCut);
```

- `cut1` y `cut2` son cortes lógicos definidos previamente.
- `combinedCut` combina ambos cortes lógicos utilizando el operador `&&`, lo que asegura que solo los eventos que cumplan ambas condiciones (energía mayor a 10 y `px` menor a 50) se seleccionen para el análisis.
- `tree->Draw("px:py", combinedCut);` Dibuja las ramas `px` y `py` solo para los eventos que cumplan con el corte combinado.

Este enfoque modular con `TCut` facilita la modificación de condiciones de corte y permite aplicar filtros de manera más eficiente y limpia en análisis más complejos.

Filtros avanzados

En situaciones de análisis de datos más complejos, es común necesitar filtrar un conjunto de datos y guardar los resultados en un nuevo archivo ROOT para su posterior procesamiento. ROOT ofrece herramientas poderosas para realizar este tipo de filtrado avanzado.

Guardar datos filtrados en un nuevo TTree

Se puede aplicar un corte a un `TTree` y guardar los eventos filtrados en un nuevo `TTree` dentro de un archivo ROOT. Este proceso es útil cuando se desea realizar análisis adicionales sobre un subconjunto específico de datos sin modificar el conjunto original.

```
TFile *outputFile = new TFile("filtered.root", "RECREATE");
TTree *filteredTree = tree->CopyTree("energy > 10 && px < 50");
filteredTree->Write();
outputFile->Close();
```

- `TFile *outputFile = new TFile("filtered.root", RECREATE);`: Crea un nuevo archivo ROOT llamado `filtered.root`, en el cual se almacenarán los datos filtrados. El modo `RECREATE` asegura que el archivo se sobrescriba si ya existe.
- `TTree *filteredTree = tree->CopyTree(energy > 10 && px < 50);`: Crea un nuevo `TTree` `filteredTree` que contiene solo los eventos que cumplen con el corte `energy > 10 && px < 50`.
- `filteredTree->Write();`: Guarda el `TTree` filtrado en el archivo ROOT `filtered.root`.
- `outputFile->Close();`: Cierra el archivo ROOT una vez que los datos han sido escritos.

Este método es particularmente útil cuando se necesita crear subconjuntos de datos específicos para realizar análisis adicionales sin cargar todo el conjunto original en la memoria.

Filtrar eventos en memoria

Para filtrar eventos durante el análisis de datos en memoria y optimizar el proceso de selección de eventos, podemos crear una lista de entradas que cumplan con un corte específico. Este enfoque es útil cuando no se desea escribir un nuevo archivo, sino que se quiere procesar solo un subconjunto de eventos de un `TTree` original, directamente en memoria.

El proceso se puede realizar mediante los siguientes pasos:

```
tree->SetEntryList(0); // Resetea la lista de entradas
tree->Draw(">>entryList", "energy > 10"); // Genera lista de entradas que
    cumplen el corte
TEntryList *entryList = (TEntryList*)gDirectory->Get("entryList");
tree->SetEntryList(entryList); // Asocia la lista de entradas al TTree

// Iterar solo sobre los eventos que cumplen el filtro
for (int i = 0; i < entryList->GetN(); i++) {
    int entry = entryList->GetEntry(i);
    tree->GetEntry(entry);
    std::cout << "px: " << px << ", energy: " << energy << std::endl;
}
```

- `>> entryList`: Esta sintaxis indica que los eventos seleccionados por el filtro se deben guardar en una lista llamada `entryList`. El `>>` es un operador especial que genera la lista de entradas filtradas directamente desde el `TTree`.
- `gDirectory->Get()`: Este comando recupera la lista de entradas generada en el paso anterior. `gDirectory` se refiere al directorio activo dentro del entorno ROOT, que en este caso almacena la lista de entradas.
- `TEntryList`: Es la clase que ROOT utiliza para almacenar los índices de las entradas que cumplen con el corte especificado. Esta lista se puede usar para realizar un procesamiento eficiente de los datos sin necesidad de volver a acceder al archivo completo.
- `tree->SetEntryList(entryList)`: Este comando asocia la lista de entradas `entryList` al `TTree`. A partir de este momento, el `TTree` solo procesará los eventos que se encuentran en la lista.
- `entryList->GetN()`: Devuelve el número de eventos que cumplen con el filtro aplicado. Este valor puede ser utilizado para determinar cuántos eventos serán procesados en el análisis.

Este enfoque es particularmente útil cuando se desean realizar análisis interactivos o rápidos sobre un subconjunto específico de datos sin la necesidad de almacenar nuevos archivos. Además, permite un procesamiento eficiente, ya que solo se analizan los eventos que cumplen con el criterio establecido.

6. Trabajo con Ramas Complejas

Los `TTree` no solo manejan variables simples, como enteros o flotantes; también son capaces de almacenar estructuras de datos más complejas, tales como arreglos, vectores, matrices y objetos definidos por el usuario. Estos tipos de datos permiten representar colecciones más sofisticadas de información, como vectores de partículas, eventos multicanal o simulaciones complejas, lo cual es común en experimentos de física de partículas y astrofísica. Al trabajar con estas estructuras complejas, se deben emplear técnicas avanzadas de acceso y manipulación de datos, que incluyen el uso de tipos como `TClonesArray`, `std::vector`, o clases personalizadas que pueden ser almacenadas dentro de un `TTree`.

Ramas con Arreglos

Cuando cada evento tiene múltiples valores asociados, podemos usar arreglos para almacenarlos en una rama. Los arreglos permiten manejar colecciones de datos con un número fijo de elementos, lo que es adecuado para representar propiedades constantes o con una cantidad predeterminada de elementos por evento.

Ejemplo: Crear un TTree con una Rama de Arreglos

En el siguiente ejemplo, creamos un `TTree` con una rama que almacena un arreglo de 5 elementos de tipo flotante, `energy`, y llenamos la rama con 100 eventos generados aleatoriamente:

```
void ramasConArreglos() {
    TFile *file = new TFile("arreglos.root", "RECREATE");
    TTree *tree = new TTree("arrayTree", "Arbol con arreglos");

    float energy[5]; // Arreglo de 5 elementos
    tree->Branch("energy", energy, "energy[5]/F");

    for (int i = 0; i < 100; i++) {
        for (int j = 0; j < 5; j++) {
            energy[j] = gRandom->Uniform(0, 10); // Asignar valores aleatorios
        }
        tree->Fill(); // Llenar arbol con datos del evento
    }
    tree->Write(); // Guardar datos en archivo
    file->Close(); // Cerrar archivo
}
```

En este código, `energy[5]` es un arreglo que contiene 5 valores flotantes que se llenan aleatoriamente en cada iteración. La rama `energy` almacena este arreglo en el `TTree`.

Leer una Rama de Arreglos

Para leer los datos de un `TTree` que contiene arreglos, utilizamos el siguiente código:

```
void leerArreglos() {
    TFile *file = TFile::Open("arreglos.root", "READ");
    TTree *tree = (TTree*)file->Get("arrayTree");

    float energy[5];
    tree->SetBranchAddress("energy", energy); // Asociar rama con arreglo

    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i); // Obtener i-esimo evento
        std::cout << "Evento " << i << ": ";
        for (int j = 0; j < 5; j++) {
            std::cout << energy[j] << " "; // Imprimir valores del arreglo
        }
        std::cout << std::endl;
    }
    file->Close(); // Cerrar archivo
}
```

En este ejemplo, el código recupera el arreglo de la rama `energy` para cada evento y lo imprime en pantalla.

Ramas con Vectores

Los vectores son más flexibles que los arreglos porque permiten un tamaño variable, lo que los hace ideales para manejar colecciones de datos que pueden cambiar entre eventos. Los vectores son una excelente opción cuando no se conoce de antemano el número de elementos por evento.

Ejemplo: Crear un TTree con una Rama de Vectores

Aquí mostramos cómo crear un `TTree` con una rama que utiliza un `std::vector<float>` para almacenar un número variable de elementos en cada evento:

```
void ramasConVectores() {
    TFile *file = new TFile("vectores.root", "RECREATE");
    TTree *tree = new TTree("vectorTree", "Arbol con vectores");

    std::vector<float> energy; // Vector para almacenar energia
    tree->Branch("energy", &energy); // Asociar rama con vector

    for (int i = 0; i < 100; i++) {
        energy.clear(); // Limpiar vector de valores previos
        int n = gRandom->Integer(10); // Numero aleatorio de elementos para el evento
        for (int j = 0; j < n; j++) {
            energy.push_back(gRandom->Uniform(0, 10)); // Agregar valores aleatorios al vector
        }
        tree->Fill(); // Llenar arbol con datos del evento
    }
    tree->Write(); // Guardar datos en archivo
    file->Close(); // Cerrar archivo
}
```

En este caso, `energy` es un `std::vector<float>` que almacena un número aleatorio de elementos para cada evento. Esta flexibilidad es útil cuando el número de elementos por evento no es constante.

Leer una Rama de Vectores

Para leer los datos almacenados en una rama que contiene un vector, utilizamos el siguiente código:

```
void leerVectores() {
    TFile *file = TFile::Open("vectores.root", "READ");
    TTree *tree = (TTree*)file->Get("vectorTree");

    std::vector<float> *energy = nullptr; // Puntero vector de energia
    tree->SetBranchAddr("energy", &energy); // Asociar rama con vector

    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i); // Obtener el i-esimo evento
        std::cout << "Evento " << i << ": ";
        for (float e : *energy) {
            std::cout << e << " "; // Imprimir valores del vector
        }
        std::cout << std::endl;
    }
    file->Close(); // Cerrar archivo
}
```

Este código muestra cómo leer un vector que se encuentra en la rama `energy` y cómo recorrer sus elementos para imprimirlos.

Manejo de Clases Personalizadas en Ramas

Los `TTree` también pueden almacenar objetos definidos por el usuario. Para ello, es necesario definir una clase con su diccionario para ROOT y crear una rama que almacene estos objetos.

Ejemplo: Clase Personalizada

Primero, definimos una clase llamada `Evento` que contiene un identificador y una energía asociada:

```
class Evento {
public:
    int id;
    float energy;
    Evento() : id(0), energy(0.0) {}
};
```

Crear y Llenar un TTree con Objetos Personalizados

El siguiente código muestra cómo crear un TTree con una rama que almacena objetos de la clase Evento:

```
void ramasConClases() {
    TFile *file = new TFile("clases.root", "RECREATE");
    TTree *tree = new TTree("classTree", "Arbol con clases");

    Evento event;
    tree->Branch("event", &event); // Asociar rama con la clase

    for (int i = 0; i < 100; i++) {
        event.id = i; // Asignar valores a miembros de la clase
        event.energy = gRandom->Uniform(0, 10);
        tree->Fill(); // Llenar arbol con el objeto
    }
    tree->Write(); // Guardar datos en archivo
    file->Close(); // Cerrar archivo
}
```

En este caso, la rama `event` almacena objetos de la clase `Evento`, y cada evento contiene un identificador y un valor de energía.

Leer una Rama con Objetos Personalizados

Finalmente, el siguiente código muestra cómo leer objetos almacenados en una rama que utiliza una clase personalizada:

```
void leerClases() {
    TFile *file = TFile::Open("clases.root", "READ");
    TTree *tree = (TTree*)file->Get("classTree");

    Evento *event = nullptr; // Puntero a objeto tipo Evento
    tree->SetBranchAddress("event", &event);

    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i); // Obtener i-esimo evento
        std::cout << "Evento " << event->id << " con energia " << event->energy <<
            std::endl;
    }
    file->Close(); // Cerrar archivo
}
```

Este ejemplo muestra cómo leer los objetos de la clase `Evento` desde el `TTree` y acceder a sus miembros para procesarlos o visualizarlos.

Tarea práctica 4

1. Filtros y cortes:

- Usa un archivo ROOT con un TTree y aplica filtros para seleccionar eventos con `energy < 10` y `px > 50`.
- Guarda los eventos filtrados en un nuevo archivo ROOT.

2. Ramas complejas:

- Crea un archivo ROOT con un TTree que tenga una rama de vectores. Llénalo con valores aleatorios.
- Lee y muestra los datos almacenados.
- Experimenta creando un TTree con una rama que almacene objetos personalizados (como la clase `Evento`).

Solución:

Parte 1: Filtrar eventos y guardar en un nuevo archivo ROOT

```
void filtrosYCortes() {  
    // Abrir archivo original  
    TFile *inputFile = TFile::Open("datos.root", "READ");  
    TTree *tree = (TTree*)inputFile->Get("eventTree");  
  
    // Crear archivo para guardar datos filtrados  
    TFile *outputFile = new TFile("filtered.root", "RECREATE");  
  
    // Filtrar eventos y copiar al nuevo TTree  
    TTree *filteredTree = tree->CopyTree("energy > 10 && px < 50");  
  
    // Guardar el nuevo TTree en el archivo  
    filteredTree->Write();  
    outputFile->Close();  
    inputFile->Close();  
  
    std::cout << "Datos filtrados guardados en filtered.root" << std::endl;  
}
```

Parte 2: Ramas complejas

```
void crearRamaDeVectores() {  
    // Crear archivo ROOT  
    TFile *file = new TFile("vectores.root", "RECREATE");  
    TTree *tree = new TTree("vectorTree", "Arbol con vectores");  
  
    // Definir un vector para almacenar datos  
    std::vector<float> energy;  
    tree->Branch("energy", &energy);  
  
    // Llenar TTree con datos aleatorios  
    for (int i = 0; i < 100; i++) {  
        energy.clear();  
        int n = gRandom->Integer(10); // Numero aleatorio de elementos en vector  
        for (int j = 0; j < n; j++) {  
            energy.push_back(gRandom->Uniform(0, 10)); // Valores aleatorios de energia  
        }  
        tree->Fill();  
    }  
  
    tree->Write();  
    file->Close();  
  
    std::cout << "Archivo vectores.root creado con exito" << std::endl;  
}
```

Leer y mostrar datos de una rama de vectores

```
void leerRamaDeVectores() {
    // Abrir archivo ROOT
    TFile *file = TFile::Open("vectores.root", "READ");
    TTree *tree = (TTree*)file->Get("vectorTree");

    // Definir un puntero al vector
    std::vector<float> *energy = nullptr;
    tree->SetBranchAddress("energy", &energy);

    // Leer datos y mostrarlos
    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i);
        std::cout << "Evento " << i << ": ";
        for (float e : *energy) {
            std::cout << e << " ";
        }
        std::cout << std::endl;
    }
    file->Close();
}
```

Crear un TTree con una clase personalizada y llenarlo

```
class Evento {
public:
    int id;
    float energy;
    Evento() : id(0), energy(0.0) {} // Constructor por defecto
};

void crearRamaDeClases() {
    // Crear archivo ROOT
    TFile *file = new TFile("clases.root", "RECREATE");
    TTree *tree = new TTree("classTree", "Arbol con clases");

    // Crear objeto de clase Evento
    Evento event;
    tree->Branch("event", &event); // Crear rama que almacene objetos Evento

    // Llenar TTree con datos
    for (int i = 0; i < 100; i++) {
        event.id = i;
        event.energy = gRandom->Uniform(0, 10); // Valores aleatorios de energia
        tree->Fill();
    }

    // Guardar TTree en archivo
    tree->Write();
    file->Close();

    std::cout << "Archivo clases.root creado con exito" << std::endl;
}
```

Leer datos de una rama que almacena objetos personalizados

```
void leerRamaDeClases() {
    TFile *file = TFile::Open("clases.root", "READ");
    TTree *tree = (TTree*)file->Get("classTree");

    // Crear puntero a un objeto Evento
    Evento *event = nullptr;
    tree->SetBranchAddress("event", &event);

    // Leer datos y mostrarlos
    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i);
        std::cout << "Evento " << event->id << ": energy = " << event->energy <<
        std::endl;
    }
    file->Close();
}
```

Cómo ejecutar la solución:

1. Guarda cada función en un archivo llamado tareasModulo4.C.
2. Ejecuta cada función en ROOT:
 - Para los filtros y cortes:

```
root -l tareasModulo4.C
root [0] filtrosYCortes()
```

- Para las ramas complejas:

```
root[0] crearRamaDeVectores()
root[1] leerRamaDeVectores()
root[2] crearRamaDeClases()
root[3] leerRamaDeClases()
```

7. Métodos avanzados en ROOT

Uso de MakeClass y MakeSelector para análisis automático

En ROOT, `MakeClass` y `MakeSelector` son herramientas automáticas diseñadas para facilitar la creación de clases y selectores a partir de un `TTree`. Estas herramientas simplifican significativamente el proceso de desarrollo de programas para el análisis de datos, proporcionando estructuras base que permiten al usuario concentrarse en los aspectos específicos de su estudio.

MakeClass

`MakeClass` genera una clase en C++ a partir de un `TTree` y su estructura, lo que permite acceder a las ramas de una manera más organizada y eficiente. Esta herramienta es especialmente útil para desarrollar programas personalizados de análisis de datos.

Ejemplo: Uso de MakeClass

Supongamos que se tiene un archivo ROOT con un `TTree` llamado `eventTree`, y se desea generar una clase que permita acceder a las ramas del árbol de forma estructurada.

```

TFile *file = TFile::Open("datos.root", "READ");
TTree *tree = (TTree*)file->Get("eventTree");

// Crear una clase a partir del TTree
tree->MakeClass("Event");

```

Este comando genera dos archivos: `Event.h` y `Event.C`. La clase `Event` creada en estos archivos incluye métodos para acceder directamente a las ramas del `TTree eventTree`.

MakeSelector

`MakeSelector` genera una clase especializada para la selección y análisis de eventos basados en criterios definidos por el usuario. Esta herramienta es particularmente útil cuando se requiere filtrar eventos y realizar cálculos específicos sobre ellos, optimizando el análisis de datos de grandes `TTrees`.

Ejemplo: Uso de MakeSelector

```

TFile *file = TFile::Open("datos.root", "READ");
TTree *tree = (TTree*)file->Get("eventTree");

// Crear un selector para analizar eventos
tree->MakeSelector("MySelector");

```

El comando anterior genera una clase llamada `MySelector`, que consta de dos archivos: `MySelector.h` y `MySelector.C`. La clase incluye el método `Process`, diseñado para ejecutarse en cada evento del `TTree`. Este método permite implementar filtros, cálculos personalizados y análisis eficiente de eventos seleccionados.

Creación de clases para estructuras de datos complejas

En ROOT, es posible crear clases personalizadas para almacenar y organizar datos complejos de forma eficiente. Estas clases permiten manejar datos más elaborados que simples arreglos o vectores, como objetos con múltiples atributos, estructuras jerárquicas o datos que requieren procesamiento especializado. Esto resulta especialmente útil en el análisis de datos de física de partículas, donde las estructuras complejas son comunes.

Ejemplo: Creación de una clase personalizada

Supongamos que queremos representar partículas con información sobre su momento lineal (`px`, `py`, `pz`) y su energía (`energy`). Esto se puede lograr definiendo una clase como la siguiente:

```

class Particle {
public:
    float px, py, pz, energy;

    Particle() : px(0), py(0), pz(0), energy(0) {}
    Particle(float _px, float _py, float _pz, float _energy)
        : px(_px), py(_py), pz(_pz), energy(_energy) {}

    // Metodo para calcular la masa
    float mass() {
        return sqrt(energy * energy - (px * px + py * py + pz * pz));
    }
};

```

- **Particle:** La clase que representa una partícula con componentes del momento (`px`, `py`, `pz`) y energía (`energy`).
- **Constructor por defecto:** Inicializa todos los atributos en cero.

- **Constructor parametrizado:** Permite inicializar una partícula con valores específicos para sus atributos.
- **Método `mass()`:** Calcula la masa de la partícula utilizando la fórmula relativista:

$$m = \sqrt{E^2 - p^2}, \quad \text{donde} \quad p^2 = px^2 + py^2 + pz^2.$$

La clase `Particle` permite encapsular propiedades y métodos específicos de una partícula, facilitando su uso en análisis más complejos. Este enfoque modular promueve un diseño limpio y reutilizable en proyectos de análisis de datos.

Uso de la clase personalizada en un `TTree`

La clase personalizada que hemos creado se puede integrar directamente en un `TTree` para organizar y almacenar datos de manera eficiente. A continuación, se muestra cómo usar la clase `Particle` en un `TTree` para almacenar información de partículas generadas aleatoriamente:

```
TFile *file = new TFile("particulas.root", "RECREATE");
TTree *tree = new TTree("particleTree", "Arbol de particulas");

Particle particle;
tree->Branch("particle", &particle);

// Llenar el TTree con datos de particulas
for (int i = 0; i < 100; i++) {
    particle.px = gRandom->Uniform(-10, 10);
    particle.py = gRandom->Uniform(-10, 10);
    particle.pz = gRandom->Uniform(-10, 10);
    particle.energy = sqrt(particle.px * particle.px +
                          particle.py * particle.py +
                          particle.pz * particle.pz + 1);

    tree->Fill();
}

tree->Write();
file->Close();
```

En este ejemplo:

- Se utiliza la clase `Particle` como plantilla para almacenar atributos relacionados con partículas individuales, como el momento lineal (`px`, `py`, `pz`) y la energía (`energy`).
- Se crea un `TTree` llamado `particleTree`, con una rama que almacena objetos de tipo `Particle`.
- Se generan partículas con valores aleatorios para sus componentes de momento (`px`, `py`, `pz`), dentro de un rango definido mediante `gRandom->Uniform`.
- La energía se calcula utilizando la relación:

$$\text{energy} = \sqrt{p_x^2 + p_y^2 + p_z^2 + m^2},$$

asumiendo que la masa $m = 1$.

- Cada objeto de tipo `Particle` se añade al árbol mediante el método `Fill()`.
- Finalmente, el `TTree` se escribe en un archivo ROOT (`particulas.root`), y el archivo se cierra.

Este enfoque demuestra cómo combinar estructuras de datos personalizadas con la potencia de ROOT para manejar y analizar grandes conjuntos de datos de manera estructurada y eficiente.

Uso de TLorentzVector para cálculos relativistas

La clase `TLorentzVector` de ROOT es una herramienta poderosa para trabajar con vectores en cuatro dimensiones, fundamentales en cálculos de relatividad especial.

Un `TLorentzVector` representa un vector 4D (E, p_x, p_y, p_z) , donde:

- E : Energía de la partícula.
- p_x, p_y, p_z : Componentes del momento lineal.

Operaciones comunes con TLorentzVector

- ****Suma de vectores de Lorentz:**** Puedes sumar dos `TLorentzVector` para obtener el total combinado de energía y momento.
- ****Cálculo de masa invariante:**** Calcula la masa invariante de un sistema a partir de la energía total E_{total} y el momento total $\vec{p}_{\text{total}}$ utilizando:

$$m = \sqrt{E_{\text{total}}^2 - \|\vec{p}_{\text{total}}\|^2}.$$

Ejemplo: Uso de TLorentzVector

El siguiente ejemplo ilustra cómo trabajar con `TLorentzVector` para calcular la masa invariante de un sistema de partículas:

```
#include "TLorentzVector.h"

void calcularMasaInvariante() {
    // Crear dos vectores de Lorentz
    TLorentzVector p1(1, 1, 1, sqrt(3));
    // Particula 1: (px, py, pz, E)
    TLorentzVector p2(2, 2, 2, sqrt(12));
    // Particula 2: (px, py, pz, E)

    // Sumar vectores
    TLorentzVector pTotal = p1 + p2;

    // Calcular masa invariante (m = sqrt(E^2 - p^2))
    double mass = pTotal.M();
    std::cout << "La masa invariante del sistema es: " << mass << " GeV/c^2" <<
        std::endl;
}
```

Explicación del ejemplo

- **Creación de objetos `TLorentzVector`:** Cada partícula se representa como un vector de Lorentz con sus componentes (p_x, p_y, p_z, E) .
- **Suma de vectores de Lorentz:** Se calcula el vector total mediante la suma de los dos vectores individuales: $p1 + p2$.
- **Cálculo de la masa invariante:** Usamos el método `M()` del `TLorentzVector` resultante para obtener la masa invariante del sistema.
- **Salida del resultado:** Se imprime el valor de la masa invariante en unidades de GeV/c^2 .

Este ejemplo destaca la facilidad con la que `TLorentzVector` permite realizar cálculos relativistas complejos, manteniendo el código claro y eficiente.

Métodos útiles de TLorentzVector

La clase `TLorentzVector` proporciona varios métodos que facilitan el acceso y la manipulación de los componentes del vector 4D (energía y momento). A continuación, se describen algunos de los métodos más utilizados:

- `M()`: Este método calcula la **masa invariante** del vector de Lorentz, que es una propiedad fundamental en relatividad. La masa invariante se calcula usando la fórmula relativista:

$$m = \sqrt{E^2 - p^2}$$

donde E es la energía y p es el módulo del momento. Este método es útil para calcular la masa de un sistema de partículas compuesto a partir de sus vectores de energía y momento.

- `E()`: El método `E()` devuelve la energía del vector de Lorentz. En un contexto relativista, esta es la energía total de la partícula, que incluye tanto la energía cinética como la masa en reposo. Este valor es crucial cuando se realizan análisis de la dinámica de partículas.
- `Px()`, `Py()`, `Pz()`: Estos métodos devuelven las componentes del momento de la partícula en las direcciones x , y y z , respectivamente. Estas componentes son esenciales para el análisis de los movimientos de partículas en el espacio tridimensional. En física de partículas, el momento es una magnitud conservada en muchas interacciones y procesos.

Estos métodos proporcionan una forma directa y sencilla de acceder a los atributos fundamentales de las partículas representadas por los objetos `TLorentzVector`. Al ser herramientas tan importantes, el dominio de estos métodos facilita la manipulación y análisis de datos en estudios de partículas relativistas.

Tarea Práctica 5: Aplicación Integral de Métodos Avanzados en ROOT

Esta tarea tiene como objetivo consolidar los conocimientos adquiridos sobre las herramientas avanzadas de ROOT mediante ejercicios prácticos. A continuación, se presentan los ejercicios organizados por tema:

1. **Uso de MakeClass y MakeSelector:** Aplica estas herramientas para trabajar con árboles y seleccionar eventos relevantes.
 - Crea un `TTree` que contenga datos de partículas, como sus componentes de momento (`px`, `py`, `pz`) y su energía (`E`). Usa `MakeClass` para generar una clase que permita acceder a las ramas del árbol.
 - Utiliza `MakeSelector` para procesar eventos que cumplan una condición específica, como seleccionar partículas con energía mayor a 10 GeV.
2. **Creación de Clases Personalizadas:** Diseña una clase que permita organizar y procesar datos de partículas.
 - Define una clase `Particle` con atributos como `px`, `py`, `pz`, `energy` y un método que calcule la masa invariante utilizando la fórmula relativista.
 - Instancia múltiples objetos de la clase `Particle` y llama al método de cálculo de masa invariante para evaluar diferentes partículas.
3. **Uso de TLorentzVector:** Experimenta con vectores de Lorentz para realizar cálculos relativistas.
 - Representa dos partículas mediante objetos `TLorentzVector` y calcula la masa invariante del sistema combinado.
 - Realiza operaciones adicionales, como la suma de vectores de Lorentz, y analiza los resultados obtenidos.

Solución:

Parte 1: Uso de MakeClass y MakeSelector

Crear un TTree con datos de partículas

```
void crearTTreeParticulas() {
    TFile *file = new TFile("particles.root", "RECREATE");
    TTree *tree = new TTree("particleTree", "Arbol de particulas");

    float px, py, pz, energy;
    tree->Branch("px", &px, "px/F");
    tree->Branch("py", &py, "py/F");
    tree->Branch("pz", &pz, "pz/F");
    tree->Branch("energy", &energy, "energy/F");

    for (int i = 0; i < 1000; i++) {
        px = gRandom->Uniform(-10, 10);
        py = gRandom->Uniform(-10, 10);
        pz = gRandom->Uniform(-10, 10);
        energy = sqrt(px * px + py * py + pz * pz + 1);
        tree->Fill();
    }

    tree->Write();
    file->Close();

    std::cout << "Archivo particles.root creado con exito" << std::endl;
}
```

Usar MakeClass

Después de ejecutar crearTTreeParticulas, abre ROOT y utiliza el comando:

```
TFile *file = TFile::Open("particles.root", "READ");
TTree *tree = (TTree*)file->Get("particleTree");
tree->MakeClass("ParticleAnalysis");
```

Esto genera los archivos ParticleAnalysis.h y ParticleAnalysis.C.

Usar MakeSelector

Para analizar eventos, usa el comando:

```
TFile *file = TFile::Open("particles.root", "READ");
TTree *tree = (TTree*)file->Get("particleTree");
tree->MakeSelector("ParticleSelector");
```

Edita el archivo ParticleSelector.C para filtrar eventos con energía mayor a 10 GeV:

```
Bool_t ParticleSelector::Process(Long64_t entry) {
    fChain->GetEntry(entry);

    if (energy > 10) {
        std::cout << "Evento " << entry << ": energia = " << energy << std::endl;
    }
    return kTRUE;
}
```

Ejecuta el selector en ROOT:

```
tree->Process("ParticleSelector.C");
```

Parte 2: Creación de clases personalizadas

Crear la clase Particle

```
class Particle {
public:
    float px, py, pz, energy;

    Particle() : px(0), py(0), pz(0), energy(0) {}
    Particle(float _px, float _py, float _pz, float _energy)
        : px(_px), py(_py), pz(_pz), energy(_energy) {}

    float mass() const {
        return sqrt(energy * energy - (px * px + py * py + pz * pz));
    }
};
```

Crear un TTree con objetos Particle

```
void crearTTreeConClases() {
    TFile *file = new TFile("particles_class.root", "RECREATE");
    TTree *tree = new TTree("particleTree", "Arbol de particulas con clase");

    Particle particle;
    tree->Branch("particle", &particle);

    for (int i = 0; i < 100; i++) {
        particle.px = gRandom->Uniform(-10, 10);
        particle.py = gRandom->Uniform(-10, 10);
        particle.pz = gRandom->Uniform(-10, 10);
        particle.energy = sqrt(particle.px * particle.px + particle.py * particle.py + particle.pz * particle.pz + 1);
        tree->Fill();
    }

    tree->Write();
    file->Close();
    std::cout << "Archivo particles_class.root creado con exito." << std::endl;
}
```

Leer datos desde el TTree

```
void leerTTreeConClases() {
    TFile *file = TFile::Open("particles_class.root", "READ");
    TTree *tree = (TTree*)file->Get("particleTree");

    Particle *particle = nullptr;
    tree->SetBranchAddress("particle", &particle);

    for (int i = 0; i < tree->GetEntries(); i++) {
        tree->GetEntry(i);
        std::cout << "Evento " << i << ": masa = " << particle->mass() << " GeV/c^2" << std::endl;
    }

    file->Close();
}
```

Parte 3: Uso de TLorentzVector

Cálculo de masa invariante

```
#include "TLorentzVector.h"

void calcularMasaInvariante() {
    // Crear dos partículas como vectores de Lorentz
    TLorentzVector p1(1, 1, 1, sqrt(3)); // px, py, pz, E
    TLorentzVector p2(2, 2, 2, sqrt(12)); // px, py, pz, E

    // Sumar los vectores
    TLorentzVector pTotal = p1 + p2;

    // Calcular la masa invariante
    double masa = pTotal.M();
    std::cout << "La masa invariante del sistema es: " << masa << " GeV/c^2" <<
        std::endl;
}
```

Cómo ejecutar los ejemplos

1. Guarda el código en un archivo llamado `tareasModulo6.C`.
2. Ejecuta cada función en ROOT:

- Para crear un TTree:

```
root -l tareasModulo6.C
root [0] crearTTreeParticulas()
```

- Para usar MakeClass y MakeSelector, sigue las instrucciones descritas.
- Para trabajar con clases personalizadas:

```
root [0] crearTTreeConClases()
root [1] leerTTreeConClases()
```

- Para usar TLorentzVector:

```
root [0] calcularMasaInvariante()
```

8. Bibliografía

- [1] *repositorio ROOT en GitHub*, <https://github.com/root-project/>.
- [2] *Introducción a ROOT Sonja Orrigo*, https://indico.cern.ch/event/1221018/contributions/5136675/attachments/2545618/4452713/root_tutorial_Orrigo.pdf.
- [3] *ROOT reference documentation*, <https://root.cern.ch/doc/master/index.html>.